

Dbms

Unit 2. Entity Relationship model

1. What is ER-Diagram? Explain its notation in detail

An **ER (Entity–Relationship) Diagram** is a conceptual data model used to visually represent the structure of a database. It was introduced by Peter Chen in 1976.

It shows:

- **Entities**
- **Attributes**
- **Relationships**
- **Constraints**

□ ER Diagram Notations

1. **Entity**
 - Represented by a **Rectangle**
 - Example: Student, Course
2. **Weak Entity**
 - Represented by a **Double Rectangle**
 - Depends on a strong entity
3. **Attribute**
 - Represented by an **Oval**
 - Types of attributes:
 - **Simple Attribute** – Cannot be divided (Age)
 - **Composite Attribute** – Can be divided (Name → FirstName, LastName)
 - **Multivalued Attribute** – Double oval (Phone_No)
 - **Derived Attribute** – Dashed oval (Age derived from DOB)
 - **Key Attribute** – Underlined (Roll_No)
4. **Relationship**
 - Represented by a **Diamond**
 - Shows connection between entities
 - Example: Student "enrolls" in Course
5. **Weak Relationship**
 - Double Diamond

6. Participation Constraint

- Single line → Partial participation
- Double line → Total participation

2. What are Keys in ER Model?

A **Key** is an attribute or set of attributes that uniquely identifies an entity in an entity set.

Types of Keys:

1. **Super Key**
 - Any set of attributes that uniquely identifies an entity.
2. **Candidate Key**
 - Minimal super key (no unnecessary attribute).
3. **Primary Key**
 - One candidate key chosen to uniquely identify entities.
4. **Composite Key**
 - Key made of more than one attribute.
5. **Foreign Key**
 - Attribute that refers to primary key of another entity.

□ Difference Between Primary Key and Candidate Key

Primary Key	Candidate Key
Selected from candidate keys	All possible minimal unique keys
Only one primary key	Can be multiple candidate keys
Cannot be NULL	Cannot be NULL
Used for identification officially	May or may not be selected

Example:

If Student has:

- Roll_No
- Email

Both uniquely identify → Both are **Candidate Keys**

If Roll_No selected → It becomes **Primary Key**

3. ER Diagram for Student Database

Entities:

- Student
- Department
- Admission
- Course

☐ **Entities and Attributes**

Student

- Roll_No (PK)
- Name
- DOB
- Address

Department

- Dept_ID (PK)
- Dept_Name

Course

- Course_ID (PK)
- Course_Name
- Credits

Admission

- Admission_ID (PK)
- Date
- Year

☐ **Relationships**

1. Student — belongs to — Department
(Many Students → One Department)
2. Student — enrolls in — Course
(Many-to-Many)
3. Student — gets — Admission
(One-to-One)

☐ **Text Representation of ER Diagram**

Student (Roll_No, Name, DOB, Address)
|

```

    | belongs_to (M:1)
    |
Department (Dept_ID, Dept_Name)

Student
    |
    | enrolls (M:N)
    |
Course (Course_ID, Course_Name, Credits)

Student
    |
    | gets (1:1)
    |
Admission (Admission_ID, Date, Year)

```

Explanation:

- A department can have many students.
- A student can enroll in many courses.
- Each student has one admission record.

4. Advantages and Disadvantages of ER Diagram

✓ Advantages

- Easy to understand
- Simple graphical representation
- Good for database design
- Reduces redundancy
- Improves communication between users and developers

✗ Disadvantages

- Not suitable for very large systems
- Cannot represent all constraints
- No implementation details
- Complex relationships become hard to read

5. Mapping Cardinality Representation

Mapping cardinality defines the number of entities to which another entity can be associated.

Types:

1. **One-to-One (1:1)**
 - One entity related to one entity
 - Example: Student — Admission
2. **One-to-Many (1:M)**
 - One department → Many students
3. **Many-to-One (M:1)**
 - Many students → One department
4. **Many-to-Many (M:N)**
 - Students enroll in many courses
 - Courses have many students

Representation:

- Written near relationship lines as 1, M, N
 - Or using crow's foot notation
-

6. Difference Between Strong Entity Set and Weak Entity Set

Strong Entity	Weak Entity
Has its own primary key	No primary key
Independent	Dependent on strong entity
Represented by single rectangle	Double rectangle
Example: Student	Example: Dependent

☐ Strong Entity

- Can exist independently

☐ Weak Entity

- Cannot exist without strong entity
 - Uses partial key
 - Identified by combination of:
 - Primary key of strong entity
 - Partial key
-

7. ER Diagram for Banking System

□ Entities

1. Customer

- Customer_ID (PK)
- Name
- Address
- Phone

2. Account

- Account_No (PK)
- Type
- Balance

3. Loan

- Loan_No (PK)
- Amount

4. Branch

- Branch_ID (PK)
 - Branch_Name
 - City
-

□ Relationships

1. Customer — Deposits — Account (M:N)
 2. Customer — Borrows — Loan (M:N)
 3. Account — Maintained at — Branch (M:1)
 4. Loan — Issued by — Branch (M:1)
-

□ Text ER Diagram Representation

```
Customer (Customer_ID, Name, Address, Phone)
    | Deposits (M:N)
Account (Account_No, Type, Balance)
    | Maintained_at (M:1)
Branch (Branch_ID, Branch_Name, City)
```

```
Customer
    | Borrows (M:N)
Loan (Loan_No, Amount)
    | Issued_by (M:1)
Branch
```

Explanation:

- One customer can have multiple accounts.
- One account can be joint (multiple customers).
- One branch maintains many accounts.
- One customer can take multiple loans.

8. Define Query Language. Explain DDL, DML, and DCL in Detail

□ What is a Query Language?

A **Query Language** is a language used to communicate with a database to:

- Create database structures
- Insert, update, delete data
- Retrieve data
- Control access

The most widely used query language is **SQL (Structured Query Language)**.

SQL is used in relational database systems like MySQL, Oracle, SQL Server, and PostgreSQL.

□ Types of SQL / Query Language Commands

SQL is mainly divided into:

1. **DDL (Data Definition Language)**
 2. **DML (Data Manipulation Language)**
 3. **DCL (Data Control Language)**
-

1 DDL – Data Definition Language

□ Definition:

DDL is used to **define and modify the structure of the database**.

It deals with:

- Tables
- Schema

- Indexes
- Constraints

☐ Important DDL Commands:

1. CREATE

Used to create database objects.

Example:

```
CREATE TABLE Student (  
    Roll_No INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Age INT  
);
```

2. ALTER

Used to modify an existing table.

Example:

```
ALTER TABLE Student ADD Address VARCHAR(100);
```

3. DROP

Deletes a table completely.

Example:

```
DROP TABLE Student;
```

4. TRUNCATE

Removes all records from a table but keeps structure.

Example:

```
TRUNCATE TABLE Student;
```

5. RENAME

Renames a table.

Example:

```
RENAME TABLE Student TO Students;
```

❑ Characteristics of DDL

- Changes database structure
 - Auto-commit (cannot rollback in most systems)
 - Affects entire table
-

2 DML – Data Manipulation Language

❑ Definition:

DML is used to **manage and manipulate data inside tables**.

It deals with:

- Insert data
 - Update data
 - Delete data
 - Retrieve data
-

❑ Important DML Commands:

1. INSERT

Adds new records.

```
INSERT INTO Student VALUES (1, 'Ali', 20);
```

2. SELECT

Retrieves data.

```
SELECT * FROM Student;
```

3. UPDATE

Modifies existing data.

```
UPDATE Student SET Age = 21 WHERE Roll_No = 1;
```

4. DELETE

Removes specific records.

```
DELETE FROM Student WHERE Roll_No = 1;
```

❑ Characteristics of DML

- Works on table data
- Can be rolled back
- Uses WHERE condition for specific records

3 DCL – Data Control Language

❑ Definition:

DCL is used to **control access and permissions in the database**.

It deals with:

- User rights
- Authorization
- Security

❑ Important DCL Commands:

1. GRANT

Gives permission to users.

```
GRANT SELECT, INSERT ON Student TO User1;
```

2. REVOKE

Removes permission from users.

```
REVOKE INSERT ON Student FROM User1;
```

❑ Characteristics of DCL

- Controls security
 - Manages user privileges
 - Protects database from unauthorized access
-

□ Difference Between DDL, DML and DCL

Feature	DDL	DML	DCL
Full Form	Data Definition Language	Data Manipulation Language	Data Control Language
Purpose	Defines structure	Manipulates data	Controls access
Works On	Tables, Schema	Records	Users
Example Commands	CREATE, DROP	INSERT, UPDATE	GRANT, REVOKE